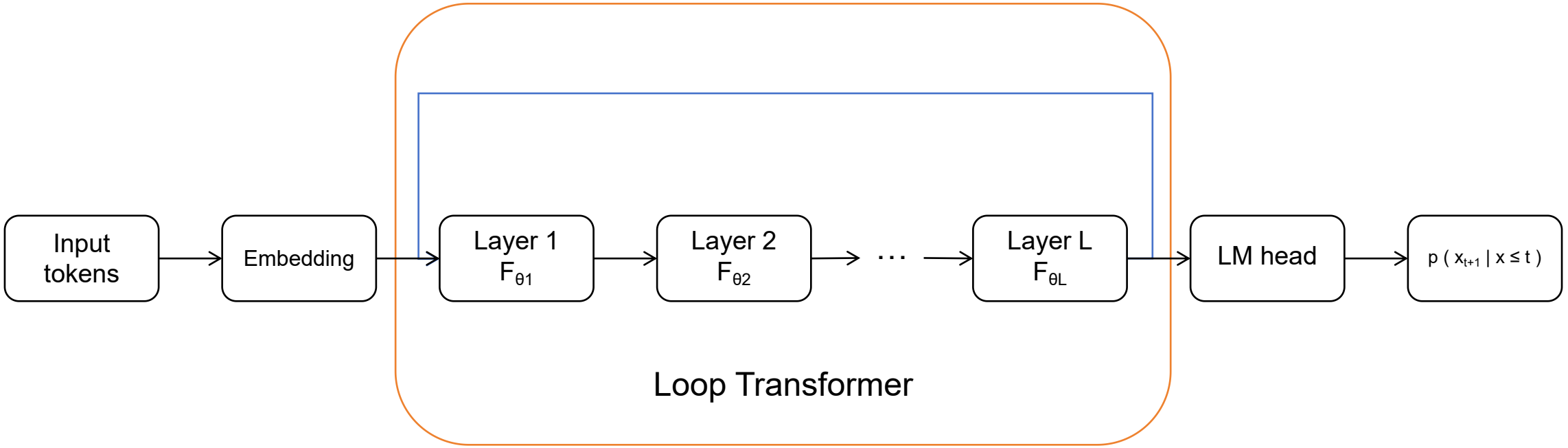
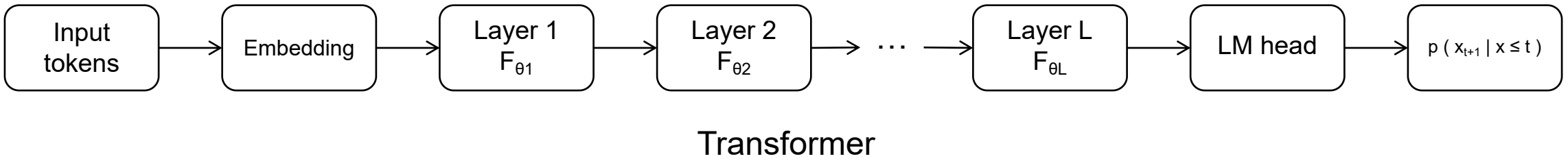
Loop Transformer

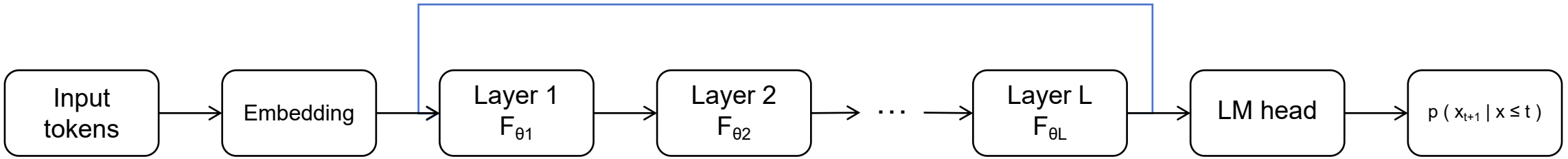
Loop Transformer



Loop Transformer

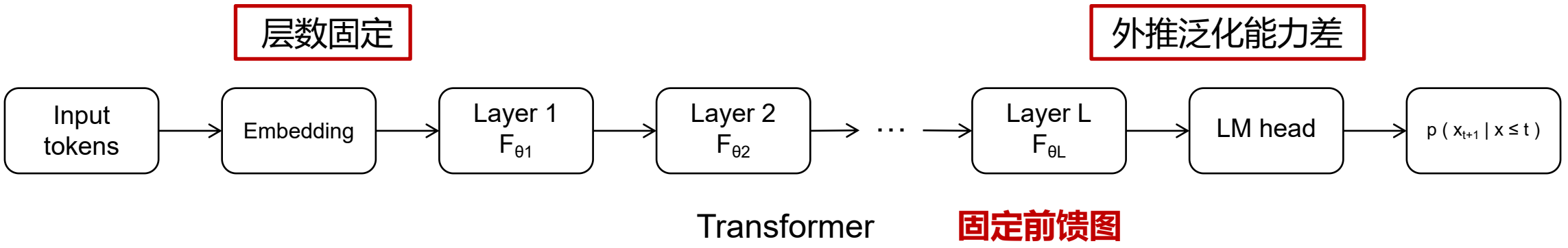


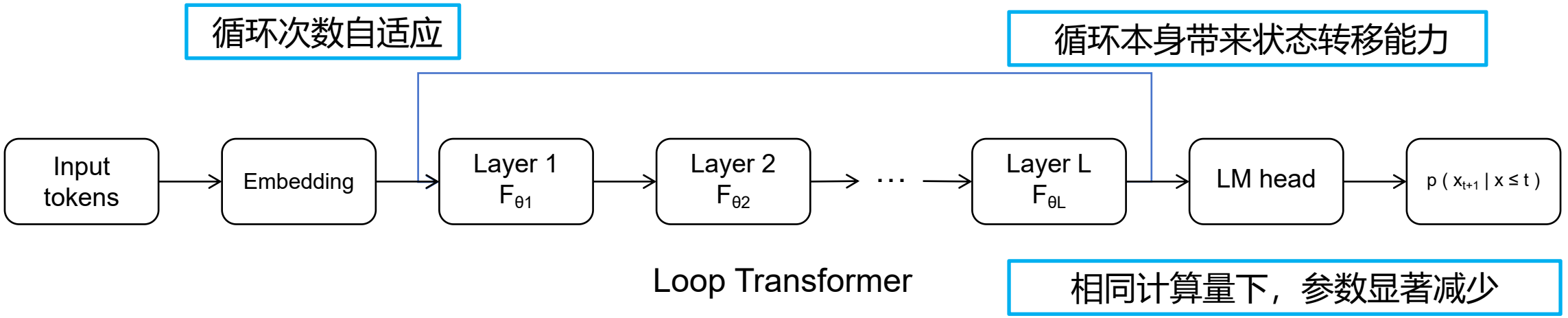
Transformer



Loop Transformer

Loop Transformer



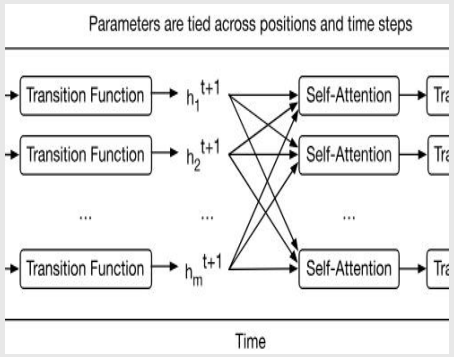


Loop Transformer

2018

Universal Transformer

Adaptive Looping.

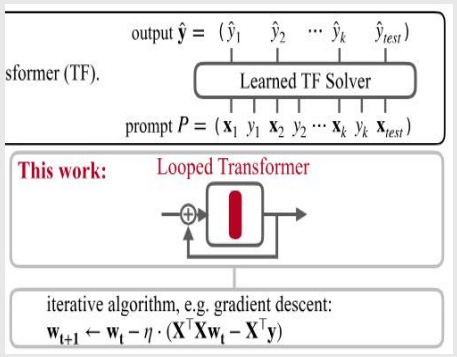


arXiv 1807.03819

2023

Algorithms

Looped Transformers 在学习算法方面表现更佳

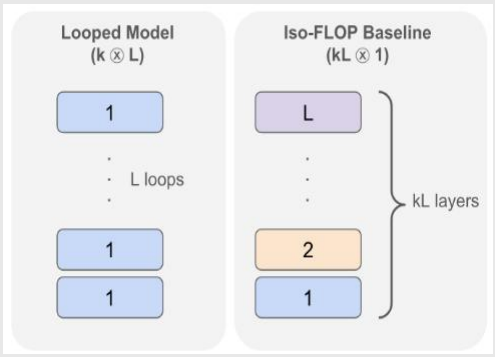


arXiv 2311.12424

2025

Latent Thoughts

基于隐形思维的推理，论Loop Transformer的能力

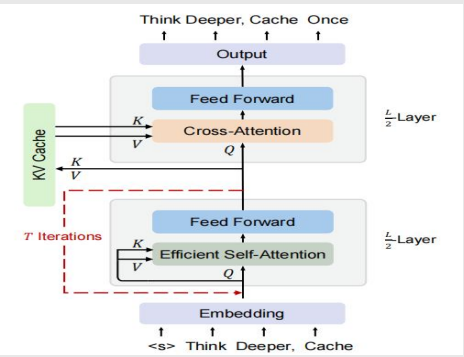


arXiv 2502.17416

2026 →

YOCO-U

兼顾参数、kv cache与模型能力的新一代架构



arXiv 2604.01220

Loop Transformer并不是新理论，它已有多年发展经历

Loop Transformer

研究重点	研究内容	代表论文	arXiv编号
核心机制	Loop Transformer机制	Universal Transformer（UT, 2018）	1807.03819
理论依据	为什么重复同一层也能增加能力?	Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach	2502.05171
循环预训练	怎样让模型学会利用多轮计算?	Reasoning with Latent Thoughts: On the Power of Looped Transformers	2502.17416
稳定性+训练效率	怎么训比较稳? 怎么训比较快?	On the Residual Scaling of Looped Transformers: Stability and Transferability	2606.18524
自适应计算	谁决定循环多少次?	LoopFormer: Elastic-Depth Looped Transformers for Latent Reasoning via Shortcut Modulation	2602.11451
部署效率	循环带来的时延和缓存问题怎么解决?	LoopCoder-v2: Only Loop Once for Efficient Test-Time Computation Scaling	2606.18023
平衡KV cache和能力	怎么能边省缓存还保持能力?	YOCO-U Universal YOCO for Efficient Depth Scaling	2604.01220

Loop Transformer

1. 什么是Loop Transformer?

2. Loop Transformer有哪些优势或缺陷?

3. 对于负载有什么影响，是否影响仿真建模?

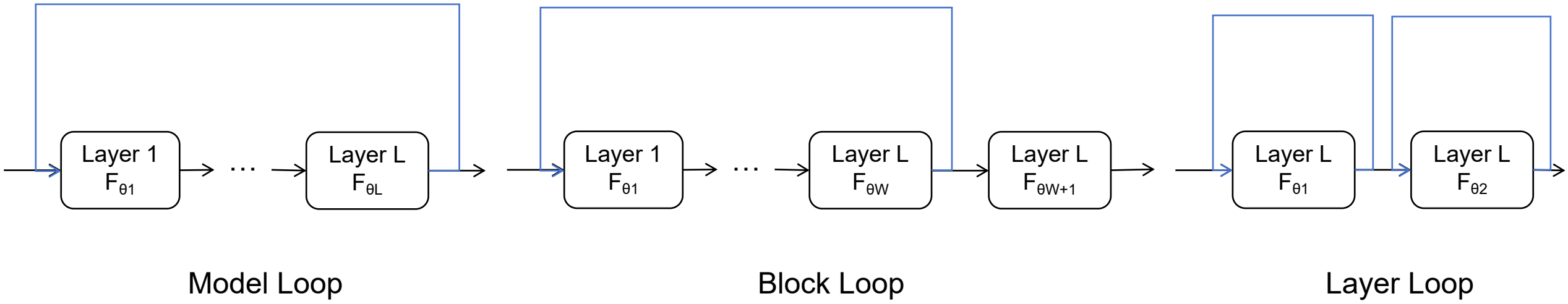
4. Loop Transformer的近期研究和发展方向

PART 1 - 基础概念

Loop Transformer，又称Looped Transformer、Recurrent-Depth Transformer

是一种通过重复执行固定 Transformer 层来增加有效计算深度的模型架构

循环模块在不同迭代中复用相同的模型参数，从而在不增加该模块参数量的情况下扩展有效计算深度



PART 1 - 基础概念

对比	Transformer	Loop Transformer
层数	L	W （通常 W << L ）
计算层数	L	W * T
总参数量	E(嵌入层等固定参数) + P * L	E + P * W
计算量	C(单层计算量) * L	C * W * T
kv cache	K(单层kv cache) * L	K * W * T

W：循环窗口大小，一次循环有几层

T：循环次数

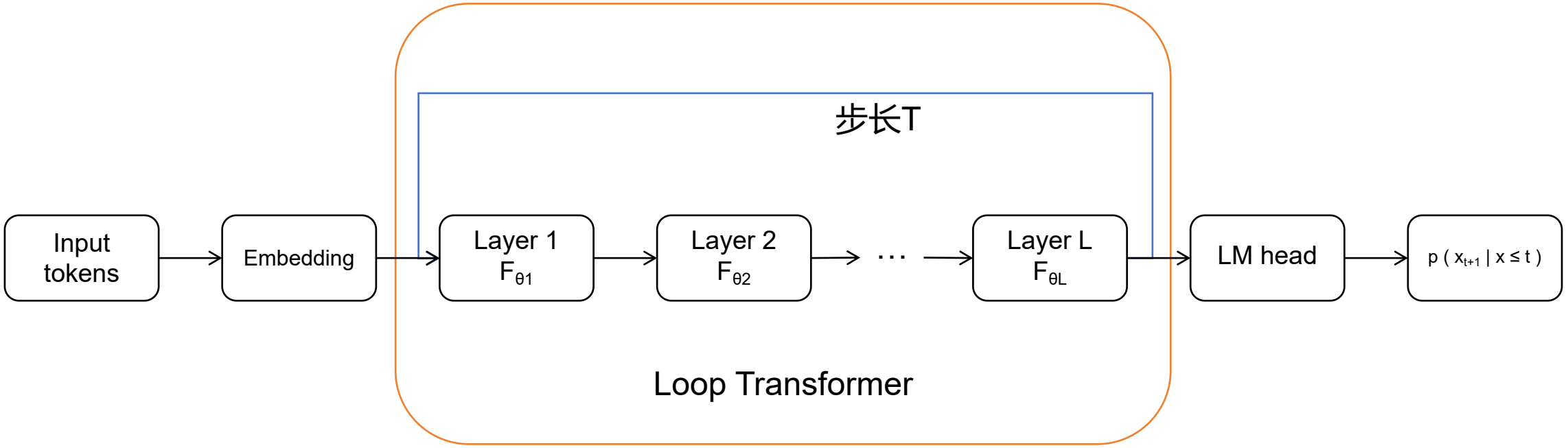
PART 1 - 基础概念

减少参数量，模型能力会不会变差？会变差多少？

	Params / FLOPs	Perplexity ↓			Language Tasks (Accuracy) ↑										
		Pile	FineWeb-Edu	OpenWebText	COPA	HS	LB	OBQA	PIQA	Race	SciQ	ARC	SIQA	WG	Avg Acc
Budget: 24x															
Base (24 ⊗ 1)	24x / 24x	**9.49**	**20.7**	**20.08**	61	**35.04**	**41.96**	27.6	**66**	29	**70.1**	**33.43**	38.18	50.4	45.27
Base-Loop (3 ⊗ 8)	3x / 24x	10.91	24.53	24.53	61	30.46	34.68	27	63.22	28.8	63.7	31.71	**38.43**	49.8	42.88
TMLT (3 ⊗ 8)	3x / 24x	10.38	22.87	21.99	65	32.34	39.06	27.8	63.11	29.67	69.8	31.67	36.95	51.54	44.69
Naive-Loop-EE (3 ⊗ 8)	3x / 24x	11.6	26.55	25.64	**66**	29.68	31.52	27	62.24	28.33	65.5	31.64	36.64	50.59	42.91
Base-Loop-EE-Cons (3 ⊗ 8)	3x / 24x	11.56	25.33	24.41	**66**	30.54	32.19	26.8	62.13	28.23	64.9	31.45	37.4	**52.88**	43.25
TMLT-EE (3 ⊗ 8)	3x / 24x	10.7	24.07	23.17	65	31.03	35.14	**28.4**	63.11	28.8	68.8	31.74	37.41	50.83	44.03
LoopFormer - Ours (3 ⊗ 8)	3x / 24x	10.28	22.87	21.98	**66**	32.3	38.27	26.8	63.33	**30.81**	68	32.71	37.97	51.94	44.81

—— LoopFormer(arXiv 2602.11451)

PART 2 - 步长


步长T
Input tokens
Embedding
Layer 1 $F_{\theta 1}$
Layer 2 $F_{\theta 2}$
…
Layer L $F_{\theta L}$
LM head
$p(x_{t+1} \mid x \le t)$
Loop Transformer

PART 2 - 步长

- 同一层的每次循环，共享参数

$$h_{k+1} = F_\theta(h_k)$$

- Learned Step Embedding，赋予共享层阶段感知能力

$$h_{k+1} = F_\theta(h_k + e_k)$$

e_k ： 第k次循环对应的可学习向量

+ 是逐元素相加

- Time + Step Size Conditioning —— LoopFormer(arXiv 2602.11451)

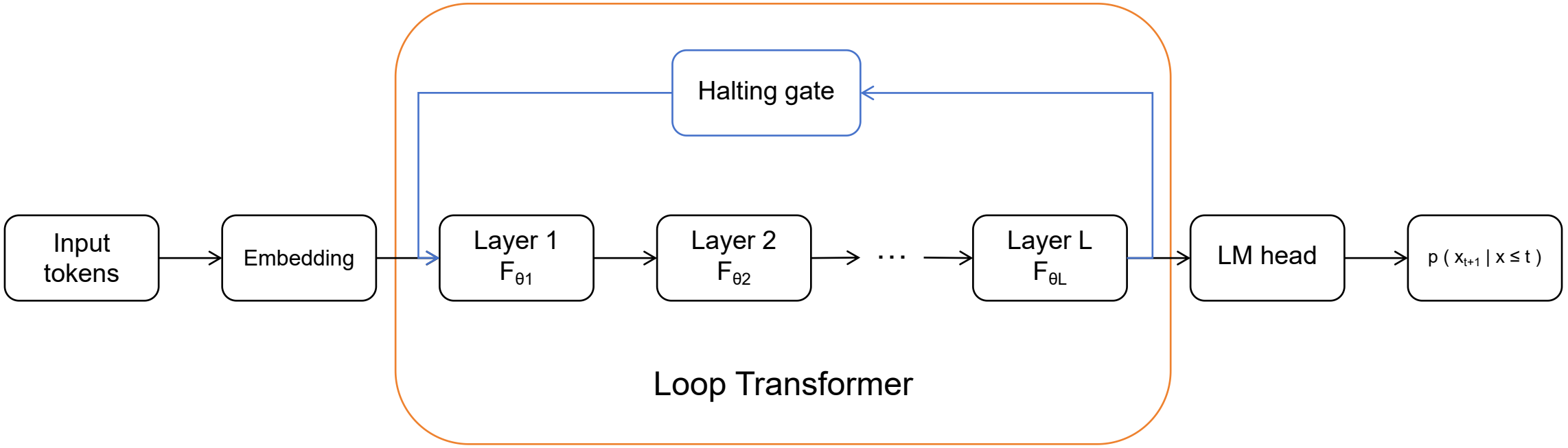
$$h_{k+1} = F_\theta(h_k,\ t_k,\ \triangle t_k)$$

t_k ： 第k次时所处的步长

通常 $t_0 = 0$, $t_M = 1$

$\triangle t_k$ ： 本次前进的步长

PART 3 - 门控


Halting gate
Input tokens
Embedding
Layer 1 $F_{\theta 1}$
Layer 2 $F_{\theta 2}$
…
Layer L $F_{\theta L}$
LM head
$p(x_{t+1} \mid x \le t)$
Loop Transformer

PART 3 - 门控

门控（Halting gate / Early-Exit）是一种打破固定循环步数，让模型能自主退出的机制

- 自适应计算时间（**A**daptive **C**omputation **T**ime） —— Universal Transformers(arXiv 1807.03819)

$$p_k = \sigma(W_h h_k + b_h)$$

$\sum p_k > 1 - \varepsilon$时退出循环

e_k ： 第k次循环对应的可学习向量

W_h、b_h：可学习参数

状态聚合

$$h_{\text{final}} = \sum_{k=1}^{N-1} p_k h^{(k)} + R \cdot h^{(N)}$$

N ： 退出时的循环次数

R ： $1 - \sum p_k$

性能问题：保留h_k占用显存严重

PART 3 - 门控

门控（Halting gate / Early-Exit）是一种打破固定循环步数，让模型能自主退出的机制

- 自适应计算时间（ACT, **A**daptive **C**omputation **T**ime）　　FLOPs

1> Cube

前向投影：2 * 隐藏维度 * 序列长度 * 步数

反向投影梯度：2 * 2 * 隐藏维度 * 序列长度 * 步数

2> Vector

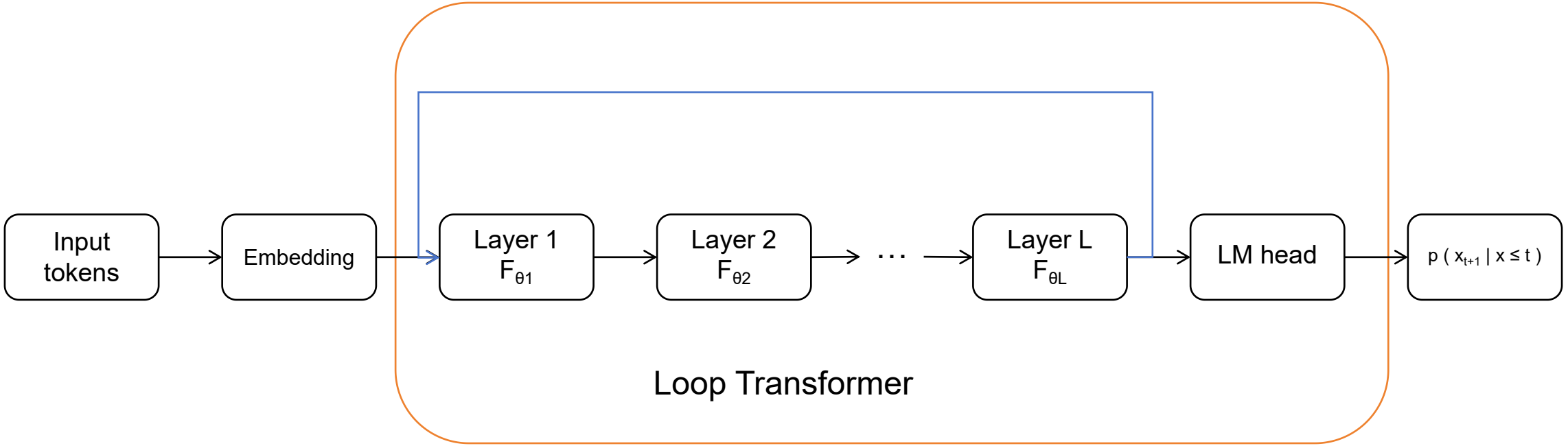
前向特征加权聚合：2 * 隐藏维度 * 序列长度 * 步数

反向特征加权回传：2 * 隐藏维度 * 序列长度 * 步数

PART 3 - 门控

门控（Halting gate / Early-Exit）是一种打破固定循环步数，让模型能自主退出的机制

- 阈值置信度门控（Confidence Thresholding） —— AMOR(arXiv 2602.13215)



PART 3 - 门控

门控（Halting gate / Early-Exit）是一种打破固定循环步数，让模型能自主退出的机制

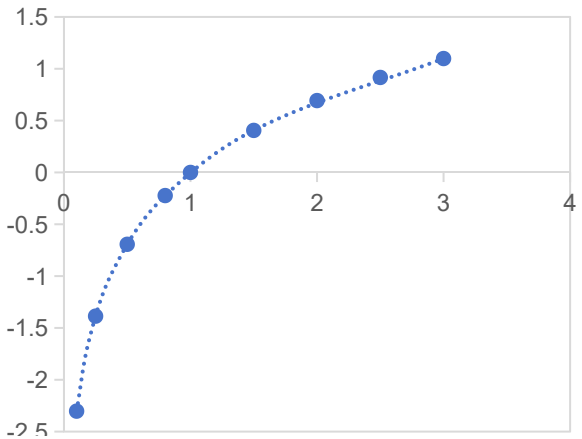
- 阈值置信度门控（Confidence Thresholding） —— AMOR(arXiv 2602.13215)

香农熵

$$\mathcal{H}\left(p^{(k)}\right) = -\sum_{i=1}^{V} p_i^{(k)} \cdot \ln\left(p_i^{(k)}\right)$$

p_i ： 词表中每个词的概率

H < τ 时，退出循环



PART 3 - 门控

门控（Halting gate / Early-Exit）是一种打破固定循环步数，让模型能自主退出的机制

- 阈值置信度门控（Confidence Thresholding） FLOPs

1> Cube

前向投影：2 * 隐藏维度 * 词表大小 * 序列长度 * 步数

反向对特征求导：2 * 隐藏维度 * 词表大小 * 序列长度 * 步数

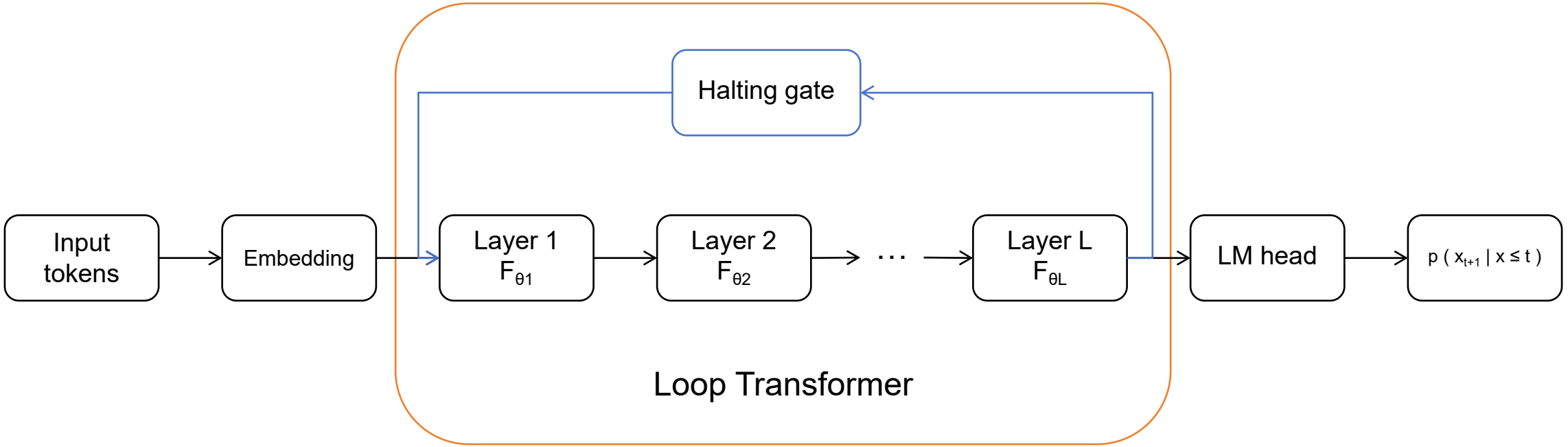
反向对权重求导：2 * 隐藏维度 * 词表大小 * 序列长度 * 步数

2> Vector

前向 Softmax：3 * 词表大小 * 序列长度 * 步数（找最大值、求 exp、求和、归一化除法）

反向交叉熵损失求导：1 * 词表大小 * 序列长度 * 最大循环步数（预测概率减去 One-hot 标签）

PART 3 - 门控



Q：推理Decode阶段，如果前一个token在第3轮退出了，下一个词需要循环4轮，找不到kv cache怎么办?

状态冻结广播：既然Token第三轮就已经收敛、确定语义后退出，那么从第4轮看，这已经是一个既定事实

采用显存虚拟指针映射，将后续轮次的kv cache映射到第3轮

PART 4 - 负载

Loop Transformer改变了参数访存模式，重复加载->持续复用

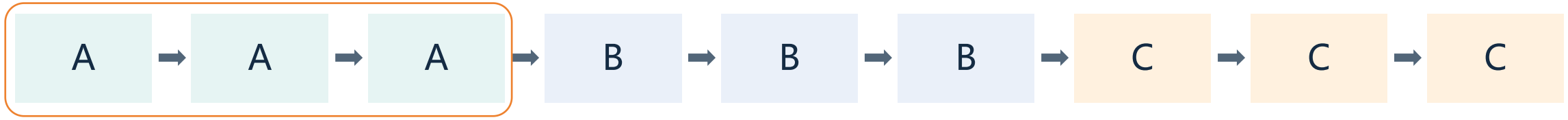
Transformer



Block Loop



Layer Loop

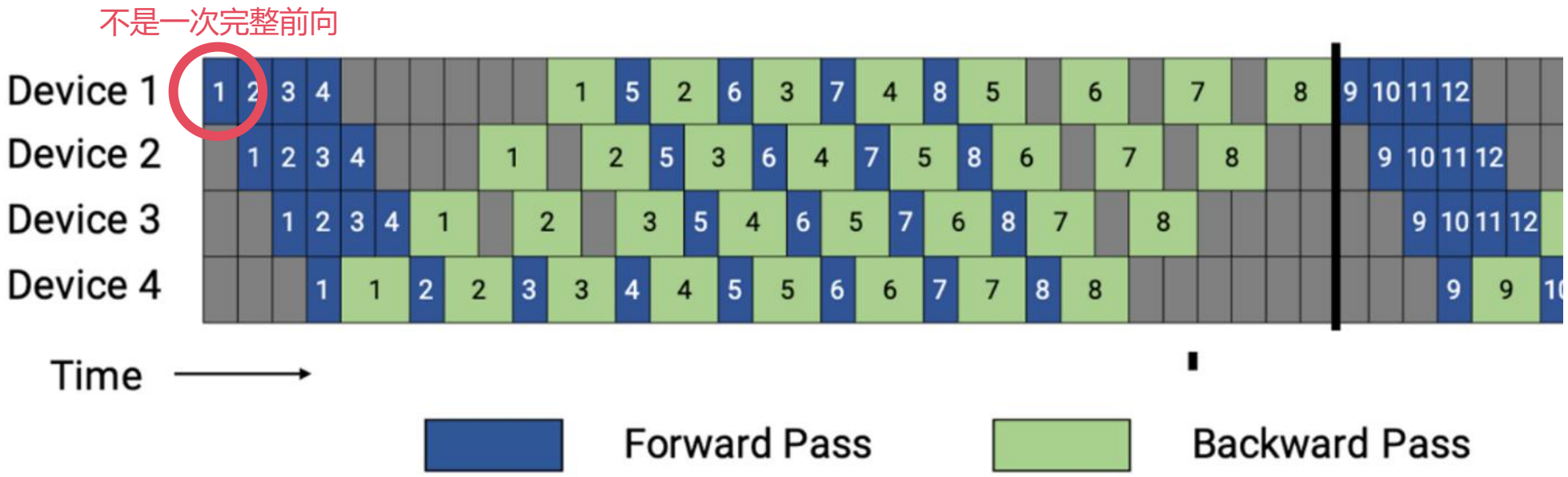


PART 4 - 负载

PP Loop Transformer对PP的影响需要分情况讨论

1. 循环完全位于一个 stage 内部， 可以将其视为更大的局部计算，仅影响stage 时长、激活容量

2. 跨 stage 的 Block Loop 破坏原始 1F1B 调度

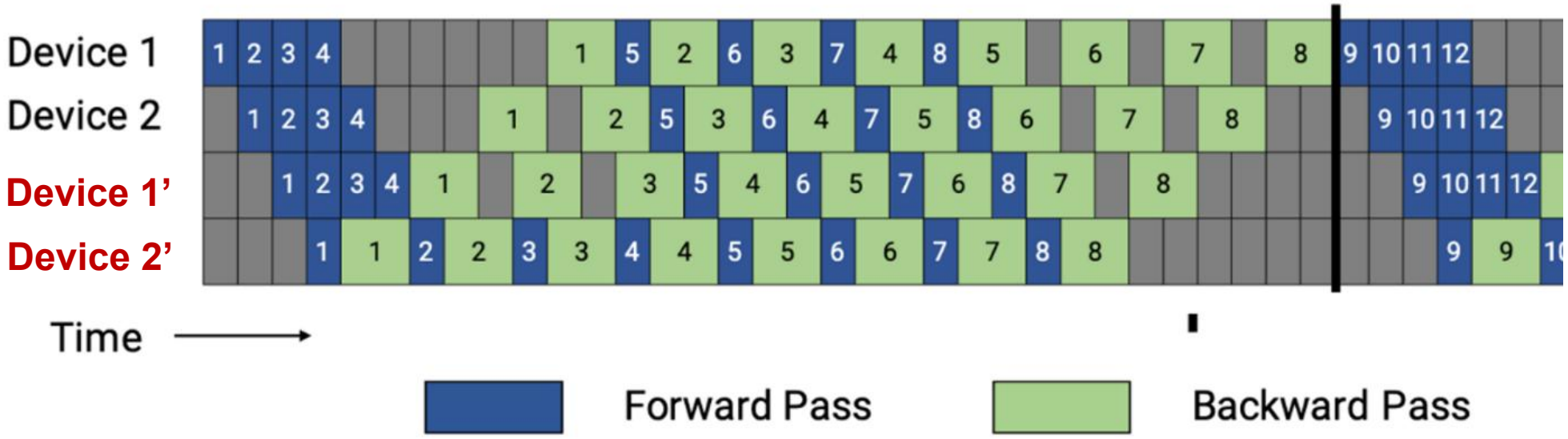


PART 4 - 负载

解决方案：展开为虚拟 stage，调度器必须识别Stage的同时，增加识别轮次编号

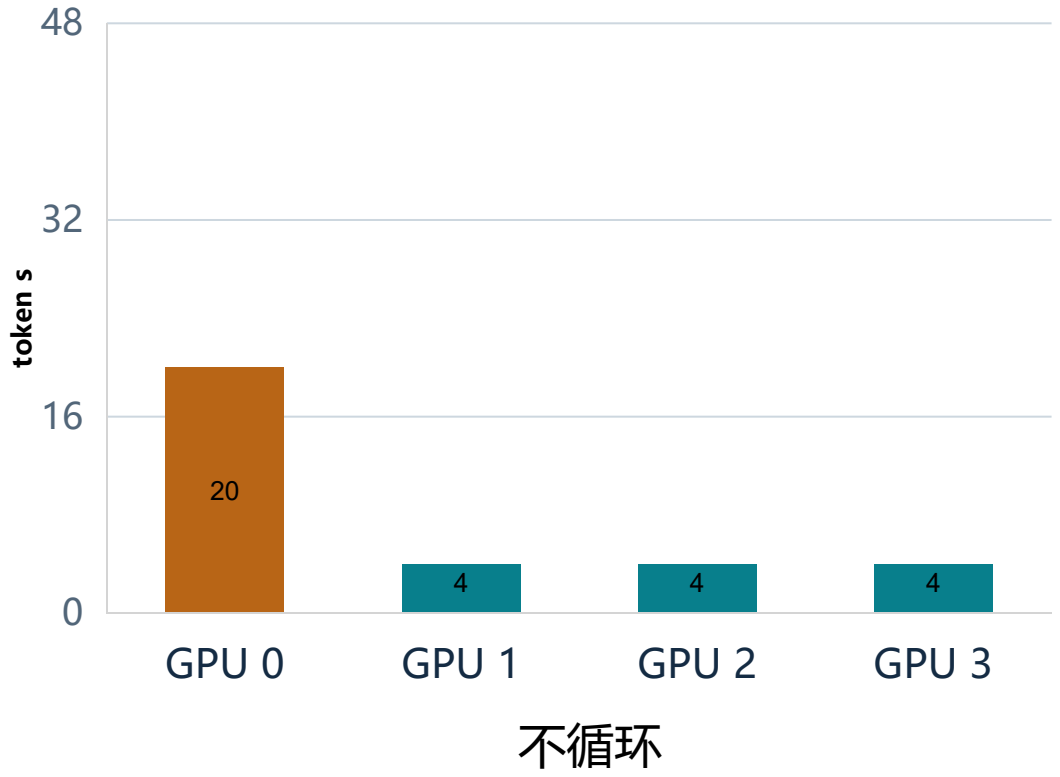
虚拟顺序：A1 -> B1 -> A2 -> B2

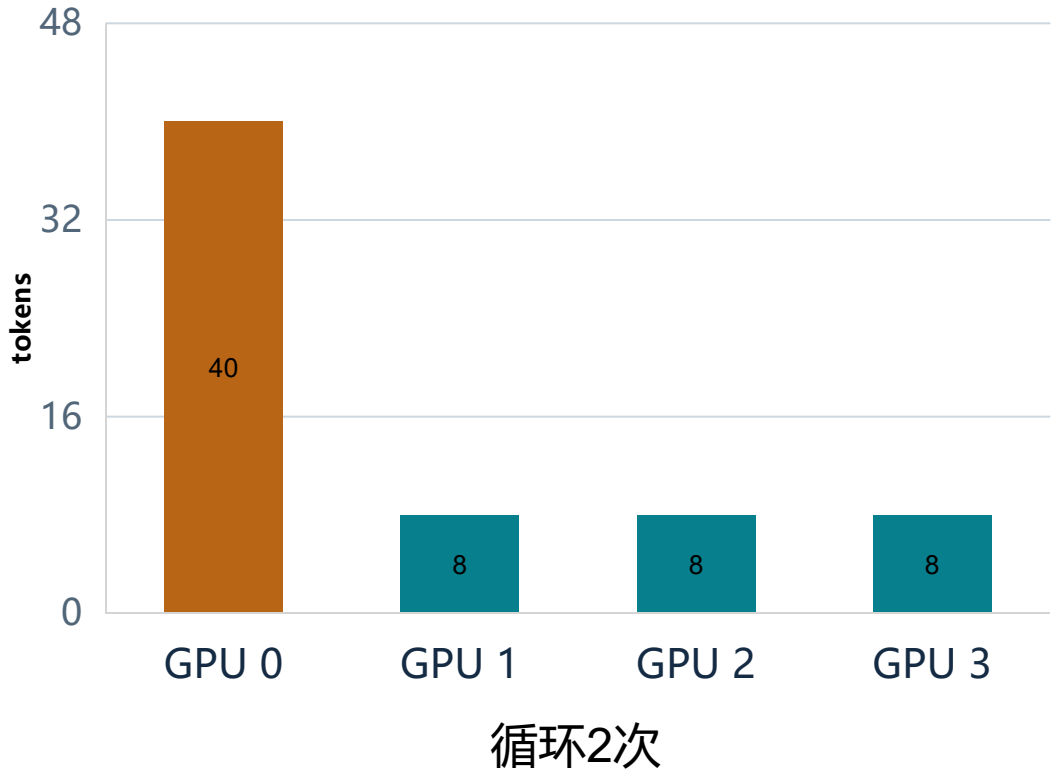
物理顺序：Device1 -> Device2 -> Device1 -> Device2



PART 4 - 负载

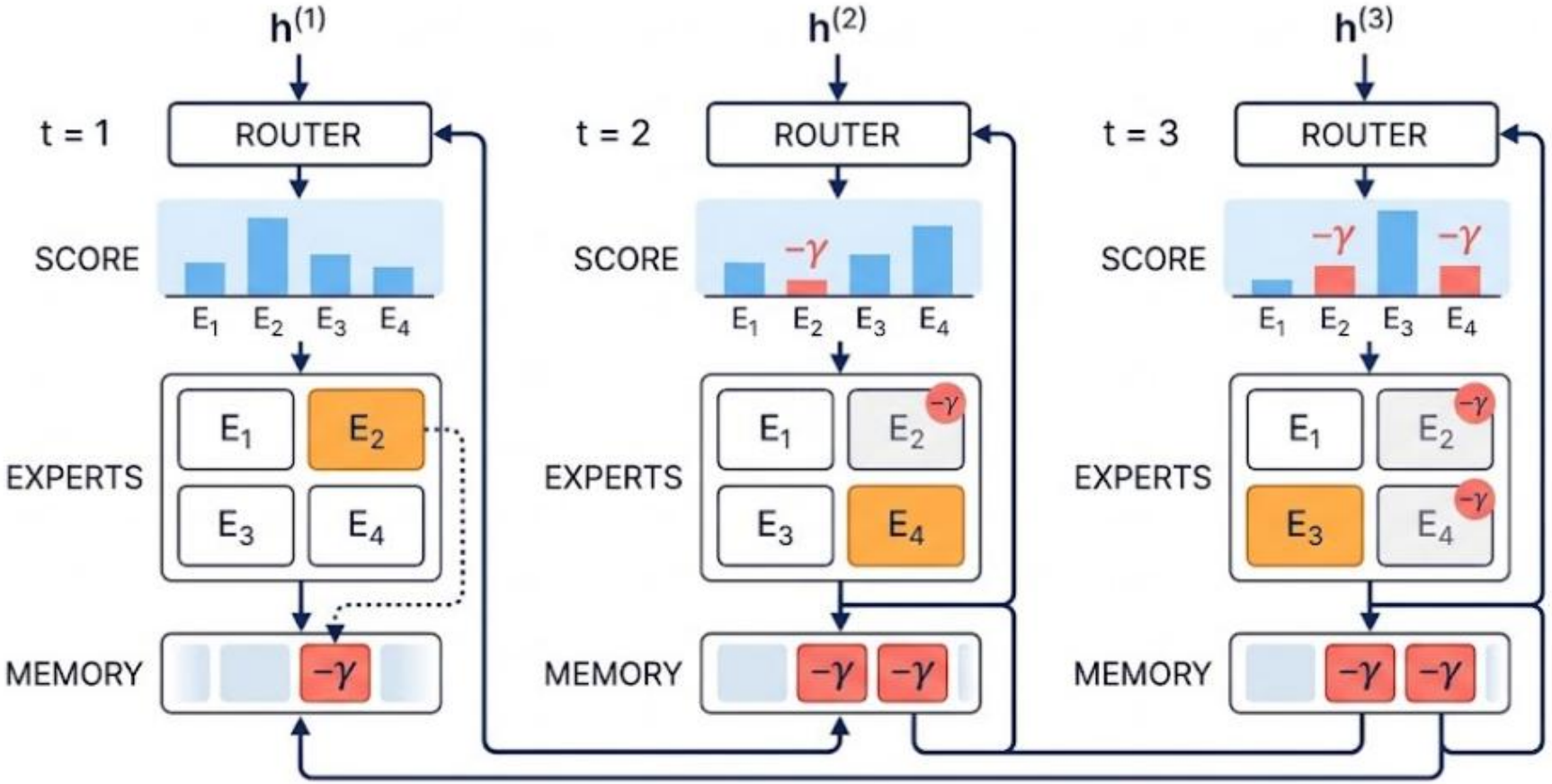
EP 循环会加深EP的不均衡，热点专家持续更久，冷门专家重复等待





PART 4 - 负载

EP 解决方案：时序疲劳偏置（Temporal Fatigue Bias）



PART 4 - 负载

EP　解决方案：时序疲劳偏置（Temporal Fatigue Bias）

Aux Loss Free:

1. 跨 Step 延迟反馈导致钟摆效应，前一轮专家很热，下一轮变得很冷
2. 它是一种作用于所有 Token 的全局标量，可能倒置“误伤”

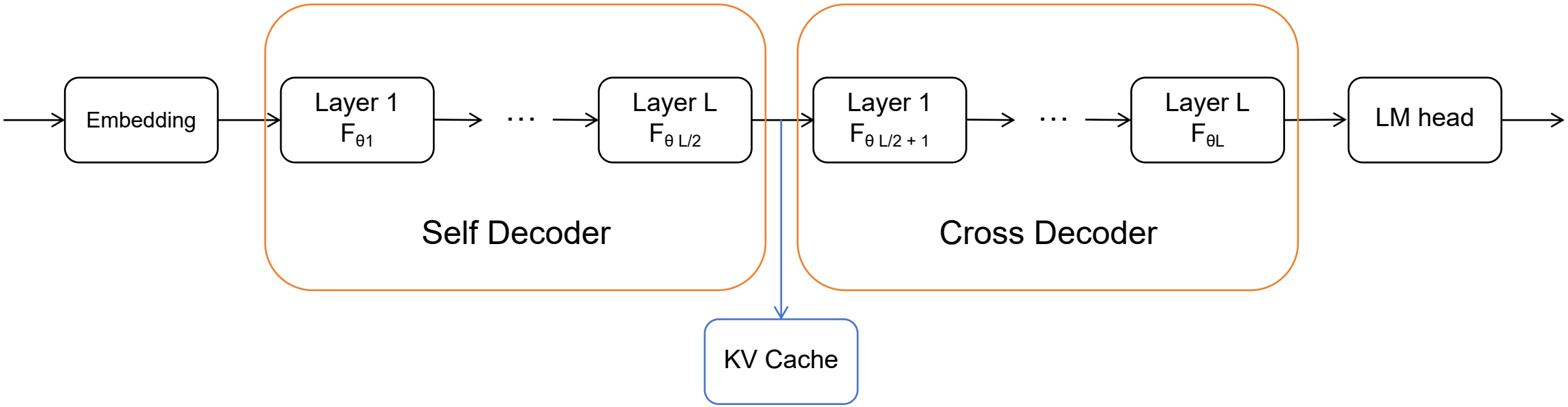
eg: E1 这个专家在浅层可能及其重要，负责进行基础表征提取，E2负责深层

强行让浅层选E2并不是一个合理的选择

YOCO: You Only Cache Once

YOCO 是一种 **Decoder-Decoder** 架构，由 **Self-Decoder** 和 **Cross-Decoder** 组成

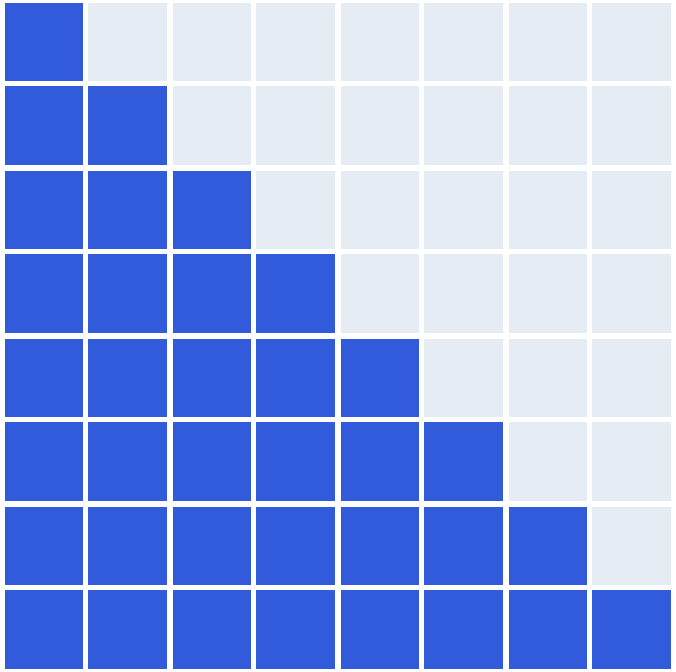
Self-Decoder 生成一份全局 KV cache，所有 Cross-Decoder 层通过交叉注意力共享读取这份缓存



YOCO: You Only Cache Once

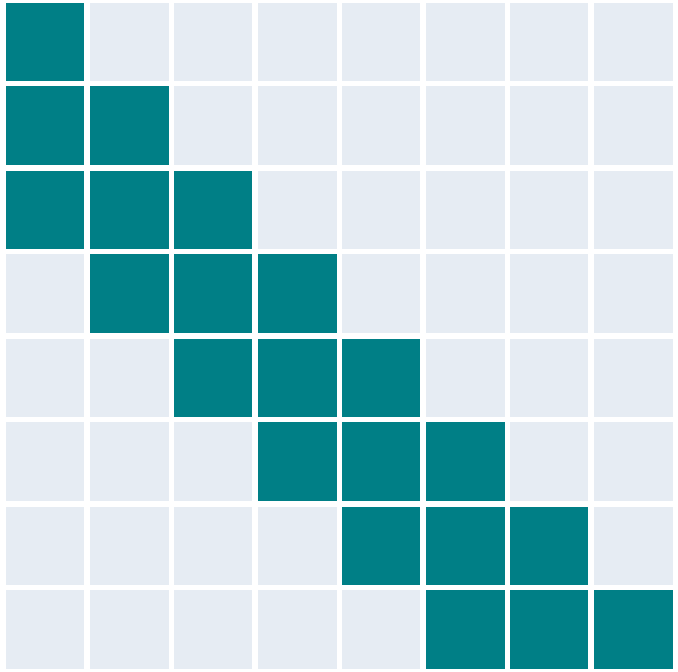
YOCO的前半段Self-Decoder采用ESA（Efficient Self-Attention）

全局因果 attention



序列长度增加，显存线性增长

ESA(SWA)

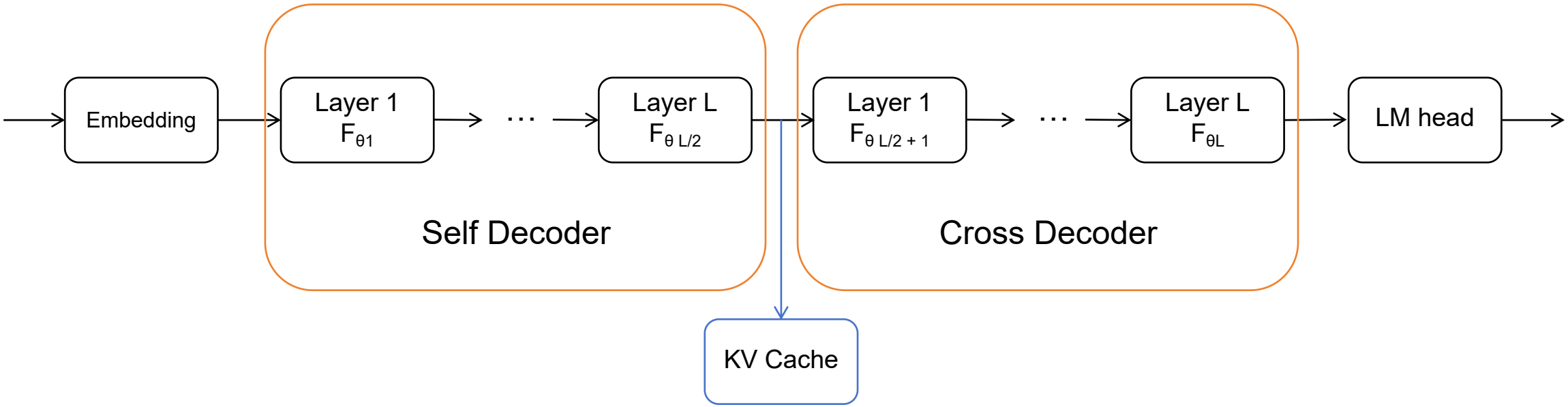


序列长度增加，显存固定不变

YOCO: You Only Cache Once

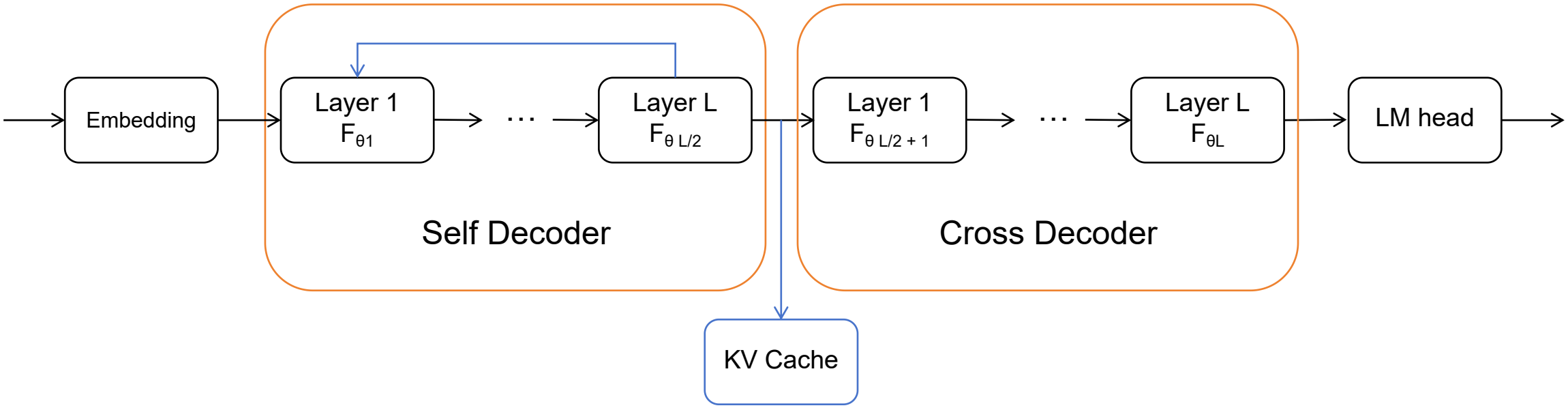
YOCO 是一种 **Decoder-Decoder** 架构，由 **Self-Decoder** 和 **Cross-Decoder** 组成

Self-Decoder 生成一份全局 KV cache，所有 Cross-Decoder 层通过交叉注意力共享读取这份缓存

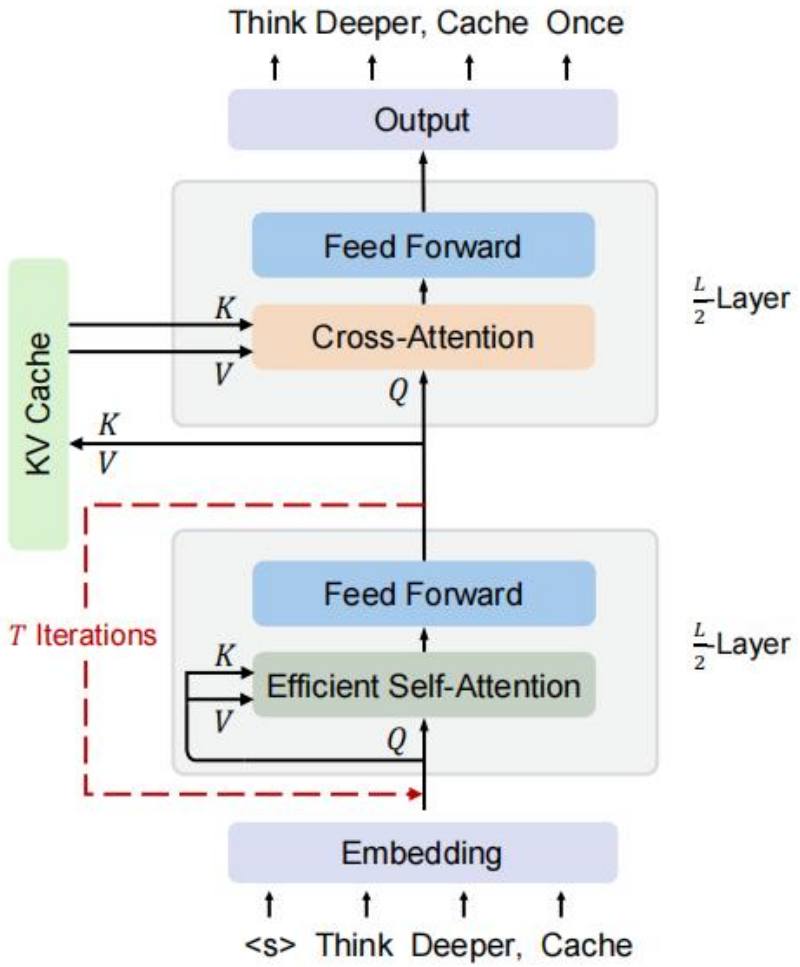


YOCO-U（Universal）

YOCO-U在YOCO的基础上增加了Self Decoder的循环



YOCO-U (Universal)



YOCO-U (Universal)

1.3B Dense 模型、20B 训练 tokens

Model	Wiki. ppl↓	LMB. ppl↓	LMB. acc↑	PIQA acc↑	OBQA acc↑	Hella. acc_n↑	Wino. acc↑	ARC-E acc_n↑	ARC-C acc_n↑	Avg. acc↑
Non-Recursive										
Transformer	22.52	22.26	38.4	**69.6**	22.6	45.7	**57.1**	59.6	36.6	47.1
YOCO	22.25	**18.30**	**41.2**	67.9	23.8	45.6	54.3	59.2	36.6	47.0
Recursive										
Loop/Universal TRM	21.56	22.56	37.7	69.0	23.6	47.7	56.3	**62.4**	38.1	47.8
ParScale	23.13	24.06	36.5	68.7	22.6	44.8	55.4	60.9	38.4	46.8
RINS	**20.98**	20.06	39.4	69.4	24.0	**49.0**	54.2	62.0	**39.9**	**48.3**
YOCO-U	21.01	18.32	**41.2**	68.7	**24.6**	48.9	55.3	62.2	37.0	**48.3**

20层

20层 * 2

10层 * 3 + 10层

YOCO

对比	Transformer	Loop Transformer	YOCO	YOCO-U
层数	L	W （通常 W << L ）	L	W + L / 2
计算层数	L	W * T	L	W * T + L / 2
总参数量	E + P * L	E + P * W	E + P * L	E + P * W + P * (L / 2)
计算量	C(单层计算量) * L	C * W * T	C * (L / 2) + c * (L / 2)	C * (W * T) + c * (L / 2)
kv cache	K(单层kv cache) * L	K * W * T	K + (L / 2) * k	K + (W * T) * k

总结

1. 什么是Loop Transformer?

   新的模型架构：循环执行Transformer层增加计算深度，分Model Loop、Block Loop、Layer Loop

2. Loop Transformer有哪些优势或缺陷?

   优势：多轮复用模型参数，参数量大幅下降　缺陷：模型能力下降，未解决kv cache问题

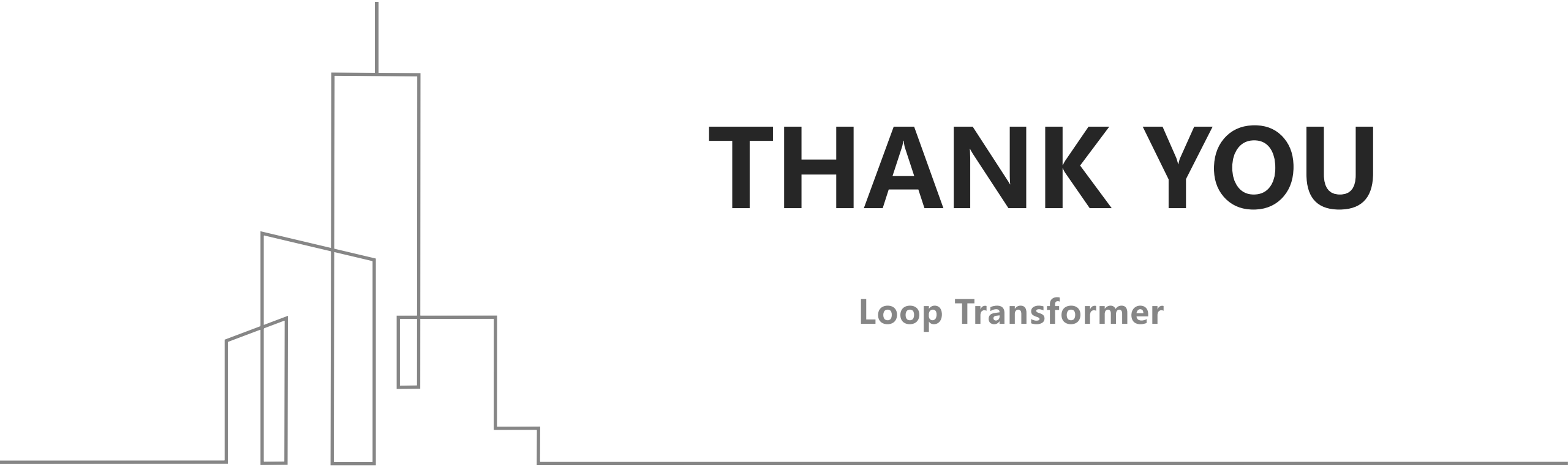
3. 对于负载有什么影响，是否影响仿真建模?

   改变了参数访存模式、跨 stage 的 Block Loop 破坏原始 1F1B 调度，需要重新适配建模、EP

4. Loop Transformer的近期研究和发展方向

   ① 多种步长、门控方案

   ② YOCO-U：解决多轮循环kv cache膨胀问题



THANK YOU

Loop Transformer